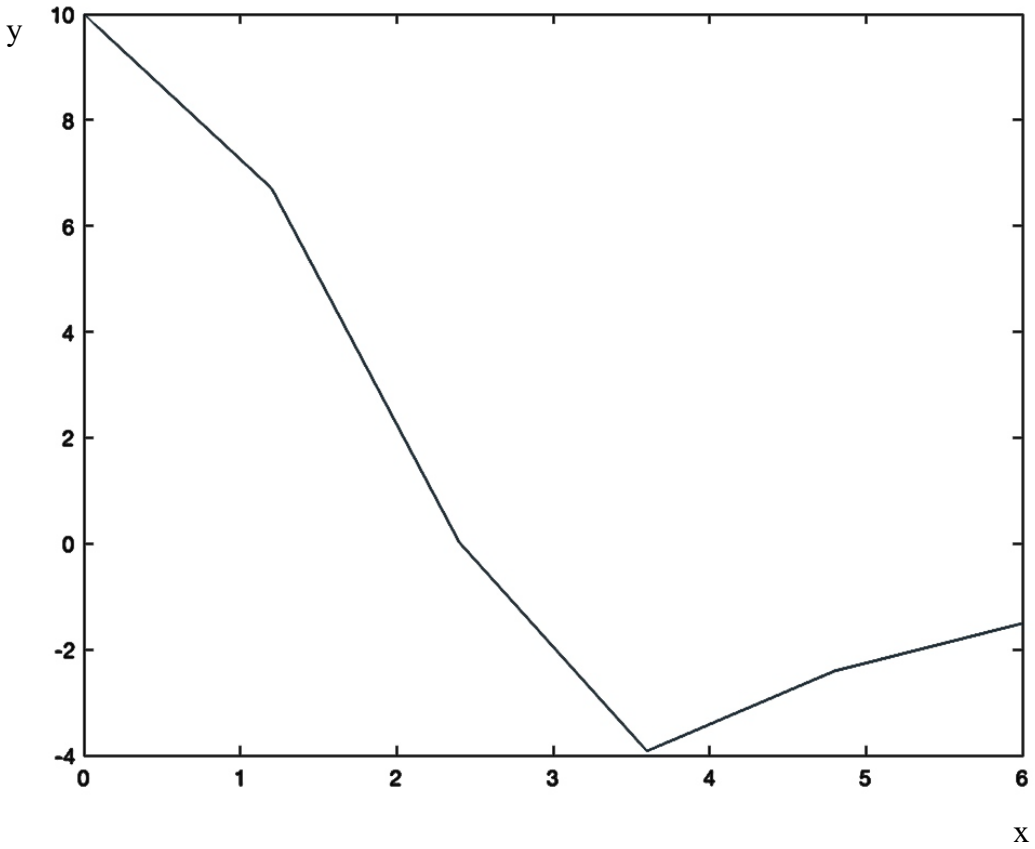
Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

MAT 214

NÜMERİK YÖNTEMLER

ÖDEV 1

1)a)



Kod:

```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 -1.5065];
plot(xvalues, yvalues );
```

b) Lagrange interpolasyon polinomu şu şekilde bulunur:

$L_{n,k} = \prod_{\substack{i=0 \\ i\neq k}}^{n} \frac{(x-x_i)}{(x_k-x_i)}$ olmak üzere

Lagrange interpolasyon polinomu = $P_n(x) = \sum_{k=0}^{n} L_{n,k}(x) * f(x_k)$

Soruda 5.dereceden Langrange interpolasyon polinomu istenilmektedir

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

O zaman Langrange İnterpolasyon polinomu aşağıdaki gibi olur:

$$P_5(x) = L_{5,0}(x) * f(x_0) + L_{5,1}(x) * f(x_1) + L_{5,2}(x) * f(x_2) + L_{5,3}(x) * f(x_3) + L_{5,4}(x) * f(x_4) + L_{5,5}(x) * f(x_5)$$

Matlaba implemente ederken $L_{n,k}$ fonksiyonunu x değerelerini alarak hesaplayan bir fonksiyon eklendi ve x 'i symbol olarak aldım böylece x değişken gibi gözükmeyecektir. Böylece P interpolasyon polinomu hesaplanırken her toplamda L fonksiyonu n ve k ya göre hesaplanıp kullanılmaktadır.
Matlab symbol değerlerini çarpıp toplayıp denkelemi sadeleştirmesi için expand() fonksiyonunu kullanıldı (langrangeip.m dosyasında)

Sonra main.m içinde x ve y vektörlerini tanımladım ve
```
lpoli = lagrangeip(xvalues, yvalues , 5);
```
çağırdım böylece fonksiyon hesaplanıp lpoli değişkenine atanmaktadır.
lpoli yaani lagrange interpolasyon polinomu sonucu bu şekilde bulundu:

P(x) = - (17855*x^5)/995328 + (24283*x^4)/165888 + (76367*x^3)/414720 - (109561*x^2)/38400 + (29129*x)/144000 + 10

Sayıların bölünmediğini farkettim ve bunun için ne yapabileceğimi araştırıldı Sayıların kesirli değil de bölünmesi için vpa() fonksiyonu kullanılabileceği bulundu. İlk parametre fonksiyon ve ikinci parametre virgülden sonra basamak sayısı olmak üzere bu fonksiyon kullanabilir ama çözünürlük düşüşü olmaması adına fonksiyon içinde kullanmamayı tercih edildi ve sadece main.m de denemeler yapmak amaçlı bırakıldı.

Virgülden sonra 10 basamak çözünürlükle bölmeler yapılırsa:
P(x) = - 0.01793881012*x^5 + 0.1463818962*x^4 + 0.1841411073*x^3 - 2.853151042*x^2 + 0.2022847222*x + 10.0
şeklinde bulunur.

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

Kodları :

Editor – lagrange.m

main.m | bisection.m | calculate_newton_raphson_error.m | lagrange.m

```
function [outputArg1] = lagrange(xvalues,n,k)
%LAGRANGE Summary of this function goes here
%   xvalues and yvalues are arrays
%   n is the degree of function
syms x real;

result = 1 ;
for i = 1:n+1 %I add 1 to k because in matlab when we parse arrays we start with 1 
    if k+1 ~= i
        result = result * ( (x-xvalues(i))/ (xvalues(k+1) - xvalues(i))  );
    end
end
outputArg1 = result;

end
```

Editor – lagrangeip.m *

main.m | bisection.m | calculate_newton_raphson_error.m | lagrange.m | lagrangeip.m *

```
function [outputArg1] = lagrangeip(xvalues, yvalues ,n)
%lagrangeip Summary of this function goes here
%   Detailed explanation goes here
syms x real;
interpolationp = 0;
for k= 0:n
    interpolationp = interpolationp + ...
        lagrange(xvalues , n , k) * yvalues(k+1);

outputArg1 = expand(interpolationp); % expand function able us to multiply
% all and summ all the similiar terms

end
```

Aşağıdaki kod main.m içindedir

```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 -1.5065];
%plot(xvalues, yvalues );
lpoli = lagrangeip(xvalues, yvalues , 5);
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

c)

$$n_k = \prod_{i=0}^{k-1}(x - x_i)$$

= newton çarpanı
Ve
$a_n = f[x_0, x_1, \dots, x_n]$
=bölünmüş farklar olmak üzere
(Köşeli parantezler bölünmüş farklara ait bir tanımdır)
Ve $f[x_i\ , x_{i+1}\ , \dots, x_{i+k}]$bu şekilde tanımlanırsa

$f[x_i] = f(x_i)\quad ve$

$$f[x_i\ , x_{i+1}\ , \dots, x_{i+k}] = \frac{f[\ x_{i+1}\ , \dots, x_{i+k}] - f[\ x_{i+1}\ , \dots, x_{i+k-1}]}{x_{i+k} - x_i}$$

Newton interpolasyon polinomu şu şekilde bulunur:

$$N(x) = \sum_{i=0}^{k} a_k n_k(x)$$

a bölünmüş farklar katsayısını bulmak için `dividedDiferences()`
fonksiyonunu yazıldı bu fonksiyon recursive bir şekilde a sayısını hesaplamaktadır.
Ve n çarpanını bulmak için newton adlı fonksiyonu yukardaki tanıma uyacak şekilde yazdım ve x'i symbol olarak tanımlaandı .
Sonra interpolasyon polinomu N(x) i bulmak için newtonip fonksiyonu yazıldı
Bu fonksiyonun içinde Newton ve dirivedDiferences fonksiyonlarına çağırarak genel fonksiyon hesaplandı ve exapand() fonksiyonuyla denklem sadeleştirildi.
main.m de Newton interplasyon polinomu bu şekilde hesaplatırıldı

```
npoli =newtonip(xvalues,yvalues,5);
```

ve sorunun cevabı ise şu şekilde bulundu:
N(x) = - (5170509988856753*x^5)/288230376151711744 + (284794035811035905*x^4)/1945555039024054272 + (39806295456412295*x^3)/216172782113783808 - (3854834990424522083*x^2)/1351079888211148800 + (85407131 2146321403*x)/4222124650659840000 + 10

Virgülden sonra 10 basamak çözünürlüğüyle bölmeler yapılırsa
N(x) = - 0.01793881012*x^5 + 0.1463818962*x^4 + 0.1841411073*x^3 - 2.853151042*x^2 + 0.2022847222*x + 10.0
sonucu elde edilir.

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

Kodları :

Editor – dividedDiferences.m *

main.m | bisection.m | calculate_newton_raphson_error.m | lagrange.m | lagrangeip.m | dividedDiferences.m *

```
function [outputArg1] = dividedDiferences(xvalues,yvalues)
%DİVİDEDDİFERENCES Summary of this function goes here
%   Detailed explanation goes here
if length(xvalues) ==1
    result = yvalues(1);
else
    result = (dividedDiferences(xvalues(2:end),yvalues(2:end)) ...
        - dividedDiferences( xvalues(1:end - 1),yvalues(1:end - 1) ) )/...
        (xvalues(end) - xvalues(1) );
end
outputArg1 = result;
end
```

Editor – newton.m

main.m | dividedDiferences.m | newton.m

```
function [outputArg1] = newton(xvalues,k)
%newton Summary of this function goes here
%   Detailed explanation goes here
syms x real;
result = 1;

for i=1:k-1
    result = result * (x - xvalues(i));
end
outputArg1 = result;
end
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

Editor – newtonip.m

main.m | dividedDiferences.m | newton.m | newtonip.m

```
function [outputArg1] = newtonip(xvalues,yvalues, n)
%NEWTONİP Summary of this function goes here
%   Detailed explanation goes here
func = 0;
for i = 1:n+1
    func = func + (dividedDiferences(xvalues(1:i) ,yvalues(1:i)) ...
        * newton(xvalues,i));
end

outputArg1 = expand(func);

end
```

```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 -1.5065];
```

```
npoli =newtonip(xvalues,yvalues,5);
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

d)
kodu :

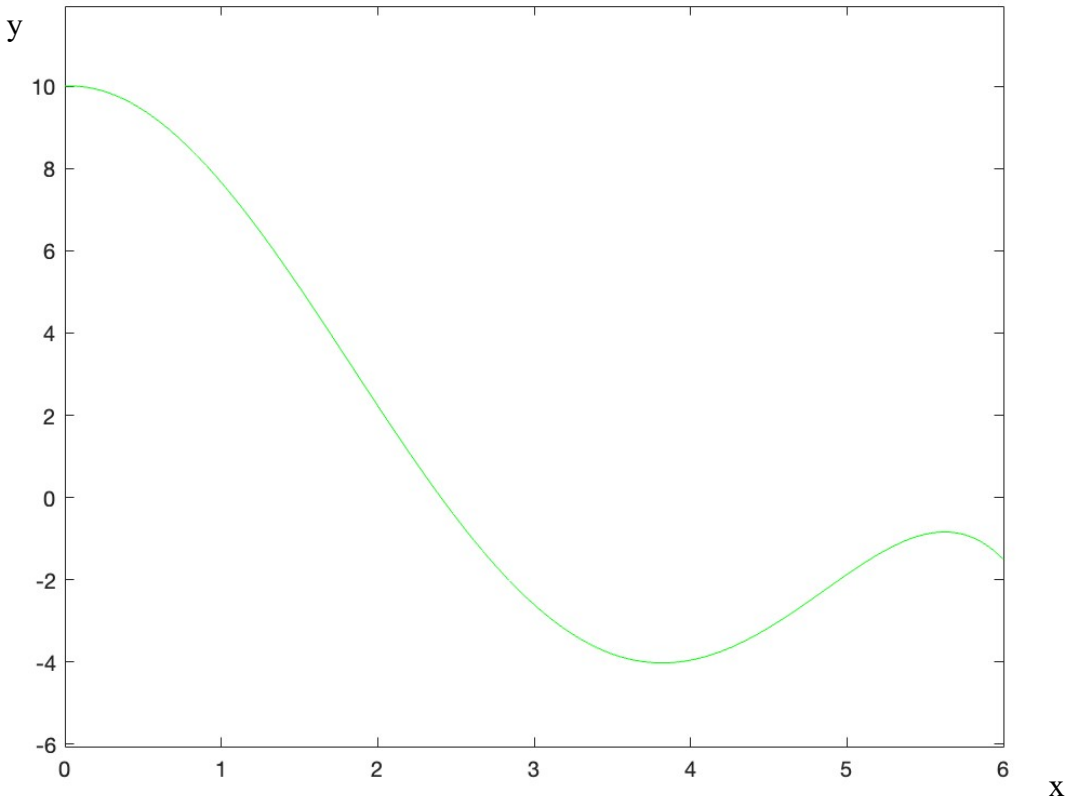
```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 -1.5065];
```

```
lpoli = lagrangeip(xvalues, yvalues , 5);
```

```
npoli =newtonip(xvalues,yvalues,5);
```

```
aax = axes;
x_cnt = 0:0.0001:6;
y_cntl = subs(lpoli, x, x_cnt);
plot(x_cnt,y_cntl)
hold on
plot(xvalues + 1,yvalues);
y_cntn = subs(npoli, x, x_cnt);
plot(x_cnt,y_cntn)
ax.ColorOrder = [1 0 0; 0 1 0; 0 0 1 ];
hold off
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808



e)

Kod:

```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 -1.5065];
```

```
lpoli = lagrangeip(xvalues, yvalues , 5);
```

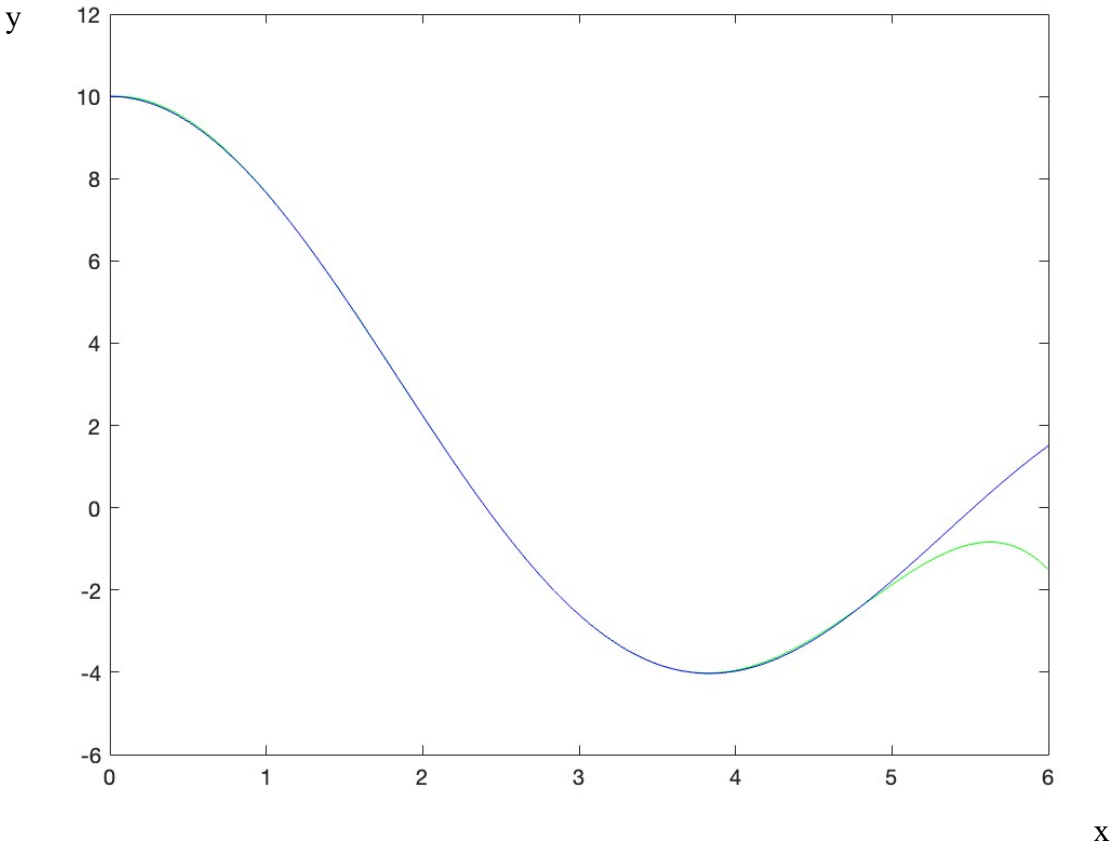
```
npoli =newtonip(xvalues,yvalues,5);
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

```
ax = axes;
%plot(xvalues,yvalues);

x_cnt = 0:0.0001:6;
y_cntl = subs(lpoli, x, x_cnt);
plot(x_cnt,y_cntl);
hold on

y_cntn = subs(npoli, x, x_cnt);
plot(x_cnt,y_cntn);
y_exact = 10 * besselj(0, x_cnt);
plot(x_cnt,y_exact);
ax.ColorOrder = [1 0 0; 0 1 0; 0 0 1 ];
hold off
```



f) Eğer değerler $f(x) = 10 * J_0(x)$ uyması gerekiyorsa ve interpolasyon polinomumuza P(x) denilirse o zaman mutlak hata

$$R(x) = |P(x) - f(x)|$$

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

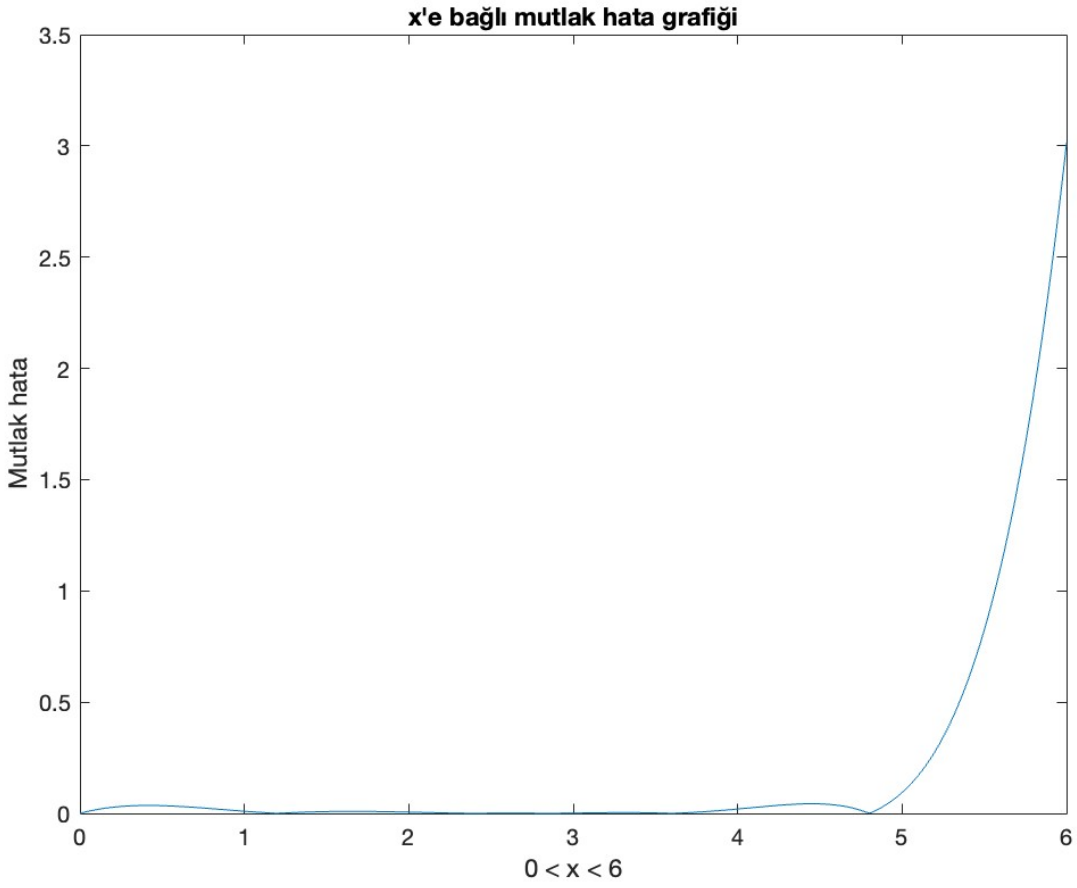
Kod:

```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 -1.5065];
```

```
lpoli = lagrangeip(xvalues, yvalues , 5);
```

```
npoli =newtonip(xvalues,yvalues,5);
```

```
27        y_l_error = abs(y_cntl- y_exact);
28        title("x'e bağlı mutlak hata grafiği")
29        xlabel('0 < x < 6')
30        ylabel('Mutlak hata')
31        plot(x_cnt , y_l_error )
```



Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

g) d şıkkında intepolasyon polinomunda x = [2 , 3] aralığında ve x = [5,6] aralığında bir kök bulunduğu gözüküyor , ama e şıkkında grafiğe bakılırsa [5,6] aralığında tam bir kök bulunamayabileceği gözükmektedir, ama [2 , 3] aralığında bir kökün bulunduğu kesindir.

h)Genel kök bulma kodları:

```
function [outputArg1,outputArg2,outputArg3,outputArg4] = bisection(f,interval , iteration_number)
%UNTITLED Summary of this function goes here
%   Detailed explanation goes here
a_all = [];
b_all = [];
p_all = [];
fp_all = [];
a = interval(1);
%a_all(end + 1) = a;
b = interval(2);
%b_all(end + 1) = b;
for i = 1:iteration_number
    p = (a + b)/2;
    a_all(end + 1) = a;
    b_all(end + 1) = b;
    p_all(end + 1) = p;
    fp_all(end + 1) = f(p);
    %xline(p);

    if f(a) * f(p)  < 0
            b = p ;
    elseif f(b) * f(p) < 0
            a = p ;
    end

end
a_all(end + 1) = a;
b_all(end + 1) = b;
p_all(end + 1) = p;
fp_all(end + 1) = f(p);

outputArg1 = a_all;
outputArg2 = b_all;
outputArg3 = p_all ;
outputArg4 = fp_all;
end
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

main.m | bisection.m | newton_raphson.m | +

```
function [outputArg1,outputArg2] = newton_raphson(f,p0,iteration_num)
%NEWTON_RAPHSON Summary of this function goes here
%   f is symbolic function
f_n = matlabFunction(f);%fn is numeric function
df = diff(f);
df_n = matlabFunction(df);

p = p0;
p_all = [p0];
fp_all = [f_n(p0)];
for i= 1:iteration_num
    p = p - (f_n(p) / df_n(p));
    p_all(end + 1) = p;
    fp_all(end+1) = [f_n(p)];
end
outputArg1 = p_all;
outputArg2 = fp_all;
end

```

main.m | bisection.m | newton_raphson.m | fixed_point.m | +

```
function [outputArg1,outputArg2] = fixed_point(fx,p0,iteration_number)
%FİXED_POİNT Summary of this function goes here
%   Detailed explanation goes here
p_all = [];
fp_all = [];
p = p0;
for i = 1:iteration_number
    p_all(end+1) = p;
    fp_all(end + 1) = fx(p);
    p = fx(p);
    %xline(p);
end
outputArg1 = p_all;
outputArg2 = fp_all;
end


```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

Editor - secant_method.m

main.m | bisection.m | newton_raphson.m | fixed_point.m | secant_method.m

```
function [outputArg1,outputArg2] = secant_method(f, p0 ,p1 ,iteration_num)
%SECANT_METHOD Summary of this function goes here
%   f is matlab function (numeric)

p_all =[p0 p1];
fp_all = [f(p0) f(p1)];
for i=3:iteration_num
    p = p_all(i-1) - ( ( f(p_all(i-1)) * ( p_all(i-1) - p_all(i-2) ) )/...
        (f(p_all(i-1)) - f(p_all(i-2)) ) ) ;
    p_all(end + 1) = p;
    fp_all(end + 1) = f(p);
end
outputArg1 = p_all;
outputArg2 = fp_all;
end
```

newton_raphson.m | fixed_point.m | secant_method.m | calculate_lagrange_error.m | calculate_newton_raphson_error.m

```
function [outputArg1,outputArg2] = calculate_newton_raphson_error(f,p0,p,inteval)
%CALCULATE_NEWTON_RAPHSON_ERROR Summary of this function goes here
%   f is symbolic function
result = (diff(diff(f))/2) * ((p - p0)^2);
delta = 0.0001;
func_result = matlabFunction(result);
value_of_result = [];
for i=inteval(1):delta:inteval(2)
    value_of_result(end + 1 ) = func_result(i);
end
numeric_result = max(value_of_result);

outputArg1 =numeric_result ;
outputArg2 = result;

end

```

Lagrange interpolasyon polinomu için kod :
bu kodları sırasıyla çalıştırarak lagrange interpolasyon polinomunun değerlerini hesapladım ve tablolara aldım (`a_it,b_it,p_it,fp_it` aynı olduğundan kullanırken diğer satırları yorum aldım)

```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 -1.5065];
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

```
lpoli = lagrangeip(xvalues, yvalues , 5);
```

```
33    lpoli_f = matlabFunction(lpoli);
34    [a_it,b_it,p_it,fp_it] = bisection(lpoli_f,[0,6],20);
35    lpoli_f_bis_t = table(a_it.',b_it.',p_it.',fp_it.');
36    l_poli_bis_err  =( b_it(end) - a_it(end)) / 2;
37    %%
38    [p_it , fp_it] = fixed_point(lpoli_f,2.5,20);
39    lpoli_f_fixed_t = table(p_it.',fp_it.');
40    %%
41    [p_it , fp_it] = newton_raphson(lpoli,2.5,4);
42    lpoli_f_newtonr_t = table(p_it.',fp_it.');
43    [l_raph_error ,l_raph_error_func] = calculate_newton_raphson_error(...
44        lpoli , p_it(end-1) , p_it(end), [2.4 , 2.5]);
45    %%
46    [p_it , fp_it] = secant_method(lpoli_f,2,3,20);
47    lpoli_f_secant_t = table(p_it.',fp_it.');
48    %%
```

1)Lagrange interpolasyon polinomu için
1.1)Bisection yöntemi ile kök bulma :

n	a_n	b_n	p_n	$f(p)_n$
0	0	6	3	-2.60189257812500
1	0	3	1.50000000000000	5.11014898681641
2	1.50000000000000	3	2.25000000000000	0.825711484909055
3	2.25000000000000	3	2.62500000000000	-1.08379267531634
4	2.25000000000000	2.62500000000000	2.43750000000000	-0.168106092674659
5	2.25000000000000	2.43750000000000	2.34375000000000	0.320399666526644
6	2.34375000000000	2.43750000000000	2.39062500000000	0.0738705619927913
7	2.39062500000000	2.43750000000000	2.41406250000000	-0.0477083371682596
8	2.39062500000000	2.41406250000000	2.40234375000000	0.0129361394776630
9	2.40234375000000	2.41406250000000	2.40820312500000	-0.0174226770663566
10	2.40234375000000	2.40820312500000	2.40527343750000	-0.00225237155271074
11	2.40234375000000	2.40527343750000	2.40380859375000	0.00533961350421386
12	2.40380859375000	2.40527343750000	2.40454101562500	0.00154305270697463
13	2.40454101562500	2.40527343750000	2.40490722656250	-0.000354801571816665
14	2.40454101562500	2.40490722656250	2.40472412109375	0.000594090040557660
15	2.40472412109375	2.40490722656250	2.40481567382813	0.000119635351339298
16	2.40481567382813	2.40490722656250	2.40486145019531	-0.000117585331155468
17	2.40481567382813	2.40486145019531	2.40483856201172	1.02445488181502e-06

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

18	2.40483856201172	2.40486145019531	2.40485000610352	-5.82805769404615e-05
		2.40485000610352	2.40484428405762	-2.86280957304541e-05
		2.40484428405762	2.40484428405762	-2.86280957304541e-05

Bisection yönteminin mutlak hatası bu şekilde hesaplanır

$$|p_n - p| \leq \frac{b_n - a_n}{2}$$

O zaman

$$|p_n - p| \leq 2.861022949218750e-06$$

1.2)Fixed point yöntemi ile (p0 = 2.5 alındı tahmini aralığın yarısı(g şıkkına bakınız))

n	p_n	$f(p)_n$
0	2.50000000000000	-0.483071508839299
1	-0.483071508839299	9.22416145324417
2	9.22416145324417	-224.565224250311
3	-224.565224250311	10614903013.2487
4	10614903013.2487	-2.41754064726611e+48
5	-2.41754064726611e+48	1.48136662916565e+240
6	1.48136662916565e+240	NaN

Bir noktaya yaklaşmak söz konusu olmadığından kök bulunamadı.

1.3)Newton raphson metodu ile (p0 = 2.5 alındı tahmini aralığın yarısı)

n	p_n	$f(p)_n$
0	2.50000000000000	-0.483071508839299
1	2.40271697595363	0.0110001983296346
2	2.40483784182635	4.75657475718094e-06
3	2.40483875970015	8.97060203897127e-13
4	2.40483875970032	0

Newton metodunun hata payı ,
epsilon tahmini kök aralığında([2.4,2.5])(fonksiyon aralığı almamıza gerek yoktur çünkü p0 = 2.5 verdik ve p_n 2.4 ve 2.5 aralığında olduğunu görüyoruz) bir sayı olmak üzere şu şekilde bulunur

$$R(p_n) = \frac{f''(\varepsilon)}{2} * (p_n - p_{n-1})^2$$

Bu hatayı bulmak için calculate_newton_raphson_error adında bir fonksiyon yazıldı içinde R(p) hata polinomunu bulundu sonra aralık sınırları içinde delta = 0.0001 olmak üzere epsilon örneklenip R(x)'in maximum olduğu nokta bulundu.

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

$$R(p) = -\frac{377186875 * x^3}{70096007590220172985161700444471296}$$

$$+\frac{102595675 * x^2}{3894222643901120721397872246915072}$$

$$+\frac{64530115 * x}{3894222643901120721397872246915072}$$

$$-\frac{55547427}{64903710731685345356631204115 2512}$$

$$\max\big(R(p)\big) = \quad 3.6424e-26$$

1.4)Sekant metodu ile (p0 =2 , p1 = 3 alındı, tahmini aralığın sınırları)

n	p_n	$f(p)_n$
0	2	2.23316255144033
1	3	-2.60189257812500
2	2.46186909799332	-0.291991389142712
3	2.39384472086108	0.0571002790687221
4	2.40497134252360	-0.000687047364209548
5	2.40483905545379	-1.53264305424727e-06
6	2.40483875969230	4.15649736851265e-11
7	2.404838759700319	1.77635683940025e-15
8	2.404838759700320	0

Secant yönteminin hatası ε hata olmak üzeri bu şekide hesaplanabilir

$$|r_{n+1} - r_n| \leq \varepsilon$$

O zaman $\varepsilon = \pm 8.8818\text{e} - 16$

Newton interpolasyon polinomu için kod :
bu kodları sırasıyla çalıştırarak Newton interpolasyon polinomunun değerlerini hesapladım ve tablolara aldım (`a_it,b_it,p_it,fp_it` aynı olduğundan kullanırken diğer satırları yoruma aldım)

```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 -1.5065];
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

```
npoli =newtonip(xvalues,yvalues,5);
```

```
%%
npoli_f = matlabFunction(npoli);
[a_it,b_it,p_it,fp_it] = bisection(npoli_f,[0,6],20);
npoli_f_bis_t = table(a_it.',b_it.',p_it.',fp_it.');
npoli_bis_err  =( b_it(end) - a_it(end)) / 2;
%%
[p_it , fp_it] = fixed_point(npoli_f,2.5,20);
npoli_f_fixed_t = table(p_it.',fp_it.');
%%
[p_it , fp_it] = newton_raphson(npoli,2.5,5);
npoli_f_newtonr_t = table(p_it.',fp_it.');
[n_raph_error ,n_raph_error_func] = calculate_newton_raphson_error...
    (lpoli , p_it(end-1) , p_it(end), [2.4 , 2.5] );
%%
[p_it , fp_it] = secant_method(npoli_f,2,3,20);
npoli_f_secant_t = table(p_it.',fp_it.');
%%
```

2)Newton interpolasyon polinomu için

2.1) Bisection yöntemi ile kök bulma :

n	a_n	b_n	p_n	$f(p)_n$
0	0	6	3	-2.60189257812501
1	0	3	1.50000000000000	5.11014898681641
2	1.50000000000000	3	2.25000000000000	0.825711484909057
3	2.25000000000000	3	2.62500000000000	-1.08379267531634
4	2.25000000000000	2.62500000000000	2.43750000000000	-0.168106092674659
5	2.25000000000000	2.43750000000000	2.34375000000000	0.320399666526646
6	2.34375000000000	2.43750000000000	2.39062500000000	0.0738705619927913
7	2.39062500000000	2.43750000000000	2.41406250000000	-0.0477083371682561
8	2.39062500000000	2.41406250000000	2.40234375000000	0.0129361394776648
9	2.40234375000000	2.41406250000000	2.40820312500000	-0.0174226770663619
10	2.40234375000000	2.40820312500000	2.40527343750000	-0.00225237155271252
11	2.40234375000000	2.40527343750000	2.40380859375000	0.00533961350421919
12	2.40380859375000	2.40527343750000	2.40454101562500	0.00154305270697108
13	2.40454101562500	2.40527343750000	2.40490722656250	-0.000354801571813113
14	2.40454101562500	2.40490722656250	2.40472412109375	0.000594090040561213
15	2.40472412109375	2.40490722656250	2.40481567382813	0.000119635351337521
16	2.40481567382813	2.40490722656250	2.40486145019531	-0.000117585331155468
17	2.40481567382813	2.40486145019531	2.40483856201172	1.02445488181502e-06
18	2.40483856201172	2.40486145019531	2.40485000610352	-5.82805769422379e-05
19	2.40483856201172	2.40485000610352	2.40484428405762	-2.86280957304541e-05

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

20	2.40483856201172	2.40484428405762	2.40484428405762	-2.86280957304541e-05

Bisection yönteminin mutlak hatası bu şekilde hesaplanır

$$|p_n - p| \leq \frac{b_n - a_n}{2}$$

O zaman

$$|p_n - p| \leq 2.861022949218750e-06$$

2.2)Fixed point yöntemi ile (p0 = 2.5 alındı tahmini aralığın yarısı(g şıkkına bakınız))

n	p_n	$f(p)_n$
0	2.50000000000000	-0.483071508839299
1	-0.483071508839299	9.22416145324417
2	9.22416145324417	-224.565224250312
3	-224.565224250312	10614903013.2489
4	10614903013.2489	-2.41754064726633e+48
5	-2.41754064726633e+48	1.48136662916633e+240
6	1.48136662916633e+240	NaN

Bir noktaya yaklaşmak söz konusu olmadığından kök bulunamadı.

2.3) Newton raphson metodu ile (p0 = 2.5 alındı tahmini aralığın yarısı)

n	p_n	$f(p)_n$
0	2.50000000000000	-0.483071508839299
1	2.40271697595363	0.0110001983296346
2	2.40483784182635	4.75657475895730e-06
3	2.40483875970015	8.97060203897127e-13
4	2.404838759700319	-1.77635683940025e-15
5	2.404838759700320	0

$$R(p_n) = \frac{f''(\varepsilon)}{2} * (p_n - p_{n-1})^2$$

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

$$R(p_n) = -\frac{89275 * x^3}{25234562732479262274658212160009666 56} + \frac{24283 * x^2}{14019201518044034597032340088894259 2} + \frac{76367 * x}{70096007590220172985161700444471296 0} - \frac{109561}{19471113219505603606989361234575360 0}$$

Ve

$$\max\big(R(p)\big) = 2.394763370407835e-31$$

2.4)Sekant metodu ile (p0 =2 , p1 = 3 alındı, tahmini aralığın sınırları)

n	p_n	$f(p)_n$
0	2	2.23316255144033
1	3	-2.60189257812501
2	2.46186909799332	-0.291991389142709
3	2.39384472086108	0.0571002790687185
4	2.40497134252360	-0.000687047364216653
5	2.40483905545379	-1.53264304891820e-06
6	2.40483875969230	4.15631973282871e-11
7	2.40483875970031	7.10542735760100e-15
8	2.404838759700319	-1.77635683940025e-15
9	2.404838759700320	0

Secant yönteminin hatası ε hata olmak üzeri bu şekide hesaplanabilir

$$|r_{n+1} - r_n| \leq \varepsilon$$

O zaman $\varepsilon = \pm 8.8818e-16$

fx interpolasyon polinomu için kod :
bu kodları sırasıyla çalıştırarak fx interpolasyon polinomunun değerlerini hesapladım ve tablolara aldım (`a_it,b_it,p_it,fp_it` aynı olduğundan kullanırken diğer satırları yoruma aldım)

```
xvalues = [0 1.2 2.4 3.6 4.8 6];
yvalues=[10.0000 6.7113 0.0251 -3.9177 -2.4043 -1.5065];
```

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

```
63    syms x real;
64    fx_poli = 10 * besselj(0,x);
65    fx_poli_f = matlabFunction(fx_poli);
66
67    [a_it,b_it,p_it,fp_it] = bisection(fx_poli_f,[0,6],20);
68    fxpoli_f_bis_t = table(a_it.',b_it.',p_it.',fp_it.');
69    fx_poli_bis_err  =( b_it(end) - a_it(end)) / 2;
70
71    [p_it , fp_it] = fixed_point(fx_poli_f,2.5,20);
72    fxoli_f_fixed_t = table(p_it.',fp_it.');
73
74    [p_it , fp_it] = newton_raphson(fx_poli,2.5,4);
75    fxpoli_f_newtonr_t = table(p_it.',fp_it.');
76    [fx_raph_error ,fx_raph_error_func] = calculate_newton_raphson_error(...
77        lpoli , p_it(end-1) , p_it(end), [2.4 , 2.5]);
78
79    [p_it , fp_it] = secant_method(fx_poli_f,2,3,9);
80    fxpoli_f_secant_t = table(p_it.',fp_it.');
81
```

3) f(x) polinomu için

3.1) Bisection yöntemi ile kök bulma :

n	a_n	b_n	p_n	$f(p)_n$
0	0	6	3	-2.60051954901934
1	0	3	1.50000000000000	5.11827671735918
2	1.50000000000000	3	2.25000000000000	0.827498512887340
3	2.25000000000000	3	2.62500000000000	-1.08487808569981
4	2.25000000000000	2.62500000000000	2.43750000000000	-0.168456529803729
5	2.25000000000000	2.43750000000000	2.34375000000000	0.320968445151562
6	2.34375000000000	2.43750000000000	2.39062500000000	0.0739378802025111
7	2.39062500000000	2.43750000000000	2.41406250000000	-0.0478608148571082
8	2.39062500000000	2.41406250000000	2.40234375000000	0.0128908822279104
9	2.40234375000000	2.41406250000000	2.40820312500000	-0.0175222207262348
10	2.40234375000000	2.40820312500000	2.40527343750000	-0.00232494022259895
11	2.40234375000000	2.40527343750000	2.40380859375000	0.00528065859637239
12	2.40380859375000	2.40527343750000	2.40454101562500	0.00147728041780734
13	2.40454101562500	2.40527343750000	2.40490722656250	-0.000423974678071470
14	2.40454101562500	2.40490722656250	2.40472412109375	0.000526616686373716
15	2.40472412109375	2.40490722656250	2.40481567382813	5.13119569749008e-05
16	2.40481567382813	2.40490722656250	2.40486145019531	-0.000186333622504870
17	2.40481567382813	2.40486145019531	2.40483856201172	-6.75113982357747e-05

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

18	2.40481567382813	2.40483856201172	2.40482711791992	-8.09986199381796e-06
19	2.40481567382813	2.40482711791992	2.40482139587402	2.16060121489915e-05
20	2.40482139587402	2.40482711791992	2.40482139587402	2.16060121489915e-05

Bisection yönteminin mutlak hatası bu şekilde hesaplanır

$$|p_n - p| \leq \frac{b_n - a_n}{2}$$

O zaman

$$|p_n - p| \leq 2.861022949218750e-06$$

3.2)Fixed point yöntemi ile (p0 = 2.5 alındı tahmini aralığın yarısı(g şıkkına bakınız))

n	p_n	$f(p)_n$
0	2.50000000000000	-0.483837764681979
1	-0.483837764681979	9.42325992933161
2	9.42325992933161	-1.80942948791555
3	-1.80942948791555	3.34502309172189
4	3.34502309172189	-3.53813386670414
5	-3.53813386670414	-3.85061519836204
6	-3.85061519836204	-4.02687510748892
7	-4.02687510748892	-3.95237940579397
8	-3.95237940579397	-3.99860477518174
9	-3.99860477518174	-3.97241584597260
10	-3.97241584597260	-3.98826205546148
11	-3.98826205546148	-3.97898758587437
12	-3.97898758587437	-3.98453268999997
13	-3.98453268999997	-3.98125694733214
14	-3.98125694733214	-3.98320632965977
15	-3.98320632965977	-3.98205121429512
16	-3.98205121429512	-3.98273744191241
17	-3.98273744191241	-3.98233038646896
18	-3.98233038646896	-3.98257206081466
19	-3.98257206081466	-3.98242865207945
20	2.50000000000000	-0.483837764681979

İstediğimizaralığın dışına değerler çıktı ve kök bulunamadı

3.3) Newton raphson metodu ile (p0 = 2.5 alındı tahmini aralığın yarısı)

n	p_n	$f(p)_n$
0	2.50000000000000	-0.483837764681979
1	2.40266676625544	0.0112123363919162
2	2.40482459179194	5.01446659545133e-06
3	2.40482555769558	1.00708169328851e-12

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

4	2.40482555769577	0

$$R(p_n) = \frac{f^{"}(\varepsilon)}{2} * (p_n - p_{n-1})^2$$

$$R(p_n) = -\frac{17048757475 * x^3}{2523456273247926227465821216000966656} + \frac{4637300227 * x^2}{14019201518044034597032340088894259 2} + \frac{14583729623 * x}{700960075902201729851617004444712960} - \frac{20922754609}{19471113219505603606989361234575360 0}$$

Ve

$$\max\big(R(p)\big) = \quad 4.5733e - 26$$

3.4)Sekant metodu ile (p0 =2 , p1 = 3 alındı, tahmini aralığın sınırları)

n	p_n	$f(p)_n$
0	2	2.23890779141236
1	3	-2.60051954901934
2	2.46263899298727	-0.296420873174667
3	2.39350783095040	0.0588931328735816
4	2.40496628922179	-0.000730582815975592
5	2.40482588615207	-1.70517254195778e-06
6	2.40482555768616	4.98985125661385e-11
7	2.404825557695772	1.57274600722102e-15
8	2.404825557695773	0

Secant yönteminin hatası ε hata olmak üzeri bu şekide hesaplanabilir

$$|r_{n+1} - r_n| \leq \varepsilon$$

O zaman $\varepsilon = \pm$ 4.4409e-16

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

i)

Fonksiyon	Bisection metodu	Fixed point metodu	Newton-raphson metodu	Secant metodu
Lagrange interpolasyon polinomu	p20 = 2.40484428405762 tolerans = 2.861022949218750e-06	Doğru kök bulunamadı (h' a bakınız)	p4 = 2.40483875970032 tolerans =3.6424e-26	p8= 2.404838759700320 tolerans = ± 8.8818e-16
Newton interpolasyon polinomu	p20 =2.40484428405762 tolerans = 2.861022949218750e-06	Doğru kök bulunamadı	p5 = 2.404838759700320 tolerans = 2.394763370407835e-31	p9 =2.404838759700320 tolerans = ± 8.8818e-16
f(x) polinomu	p20 = 2.40482139587402 tolerans =2.861022949218750e-06	Doğru kök bulunamadı	p4=2.40482555769577 tolerans = 4.5733e-26	p8=2.404825557695773 tolerans = ± 4.4409e-16

Yukardaki tablodan Newton metodunun kökü en hızlı şekilde bulduğunu görebiliriz , mesela newtonun dördüncü iterasyonunda bulunan kök secant metodu ile 8.inci iterasyonda bulunmaktadır, bisection metodu ise daha fazla iterasyonda bulmaktadır. Ama unutulmaması gereken bir şey var Newton metodun da p0'a 2.5 değeri verildi kökten daha uzak bir değer verilmiş olasaydı daha fazla iterasyona ihtiyaç duyardı, ve bisectionda eğer fonksiyon aralığı değil de daha fazla köke yakın aralıkta bir aralık verilseydi daha hızlı bulabilirdi, yine de Newton-raphsonun yaklaşma hızı sekant yönteminden büyüktür, sekant yönteminin yaklaşma hızı ise bisectiondan büyüktür.
Sabit nokta metoduna gelince fonksiyonda bir sabit noktaya yaklaşma olmadı onun için kök bulunamadı, böylece sabit nokta iterasyonun sadece spesifik problemlerde uygulanabileceğini görürüz ve bizim fonksiyonumuzda kullanılmaya uygun değil.
j) Metodların avantajları ve dezavantajlarını öğrendim:
Newton metodunun avantajı köke hızlı yaklaşmasıdır buna karşın isterleri bir başlangıç noktası ve fonksiyonun türevi , her zaman fonksiyonun türevini bulmak mümkün olmayabilir bunun için nümerik türev kullanılabilir , bu ihtiyacı kaldırmak için sekant yöntemi geliştirilmiş ve fonksiyonun türevini kullanmadan bir birine yakın iki nokta seçerek (p0 , p1) kökün bulunması

Ad-soyad : Fouad Aladhami
Öğrenci no : 210102002808

sağlanmaktadır buradaki ihtiyacımız ise ikinci bir noktadır ve genellikle bu ihtiyacı sağlamak mümkündür. Bisectionda ise tahmini bir aralık vermekteyiz Newton ve sekant metodundan daha yavaş ama bir türeve ihtiyaç duyulmamaktadır, sekant yöntemiyle ile çözünemeyen problemler daha fazla işlem yüküne katlanarak bisection ile çözülebilir. Ayrıca MATLAB kullanmayı , fonksiyon , vektör ve tablo oluşturmayı ve onlarla işlem yapmayı öğrendim.